

Evolving Network Topology in Policy Gradient Reinforcement Learning Algorithms.

1st Abhijit Banerjee,*

Dept. of Electronics and Communication
Dr. B C Roy Engineering College,
Durgapur, India
abhijit.banerjee@bcrec.ac.in

2nd Dipendranath Ghosh

Dept. of Computer Science
Dr. B C Roy Engineering College,
Durgapur, India
dipen.ghosh@bcrec.ac.in

3rd Suvrojit Das

Dept. of Computer Science and Engineering
National Institute of Technology,
Durgapur, India
suvrojit.das@cse.nitdgp.ac.in

Abstract—In this study we examine the influence of incorporating Evolutionary Algorithms (JADE) and Metaheuristic Algorithms (PSO) for evolving neural network topologies in simple policy gradient, A2C and Deep Deterministic Policy Gradient (DDPG) reinforcement learning. A2C is experimented on a discrete action space model free environment followed by DDPG on a continuous action model free environment; "LunarLanderContinuous-v2" from OpenAI gym. This baseline result is then compared with topology and hyperparameter optimized A2C and DDPG using Differential Evolution and Particle Swarm Optimization. The results thus obtained shows that Differential Evolution (JADE) optimized DDPG has the fastest convergence and minimum forgetting of learned weights.

Keywords—Deep Deterministic Policy Gradient (DDPG), Differential Evolution, Neuro-Evolution, OpenAI gym, Lunar Lander, JADE, PSO

I. INTRODUCTION

A deep model of the artificial neural network with multiple hidden layers often dubbed as Deep Neural Networks is primarily trained by gradient based algorithms. The most popular and successful such gradient based algorithm is the Backpropagation Algorithm which uses the Stochastic Gradient Descent approach for weight optimization. Computation power surge in modern days has seen the advent of multicore CPUs and GPUs which were crucial in computing the matrix multiplication in weight update of neural networks with millions of weights. This was crucial in classification or regression type problems (supervised) or even in recursive problem solving as stated by various publications [1],[2],[3].

In spite of the success of DNN in supervised learning, its application in Reinforcement Learning is quite challenging. Reinforcement Learning is the type of learning in which an agent interacts with the environment to learn a policy to maximize the cumulative rewards for the problem statement. Deep Reinforcement Learning algorithms were devised and used recently in both discrete and continuous state and action domains like, continuous space robotics[4], ATARI[5]. Such deep neural networks were effective in both discrete and continuous action space reinforcement learning.

The reported success of these algorithms is however based on the assumption or pre condition that all the hyperparameters are pre-tuned and not dynamic. The network topology

and other hyperparameters are pre-tuned to fit the problem statement. This process of hyperparameter tuning is laborious and time consuming. Stochastic Gradient Descent can find the global optima if provided sufficient runtime and enough computing power and of course if and only if a gradient exists. Several gradient descent variants have also shown similar success. Evolutionary Algorithms like Differential Evolution and its variants [6] or [7] or Metaheuristic Algorithms like Cuckoo Search [8] or PSO [9] on the other side can also be applied for weight optimization. EA finds the approximate of the gradient while SGD finds the exact gradient. Introduction of ReLU (Rectified Linear Units) as activation function shows a marked improvement in gradient flow among layers while backpropagating[10]. But, the curse of dimensionality so poorly handled by EAs will not suffice the purpose.

SGD has its own demerits too. The complexity of a Deep RL approach lies in:

- High dimensionality of action space.
- High dimensionality of observation space.
- Complicated dynamics of the real life system often ignored in toy problems.
- Too sensitive to hyperparameter changes affecting repeatability.
- Often suffers in low/sparse reward environments.

The tuning of hyperparameters and topology selection is not a straight forward task and requires experience as well as domain knowledge. This is where EAs can help to optimize the hyperparameters as well as the topology thus completely eliminating the need to hand tune the hyperparameters.

In this study, we concentrate on three policy gradient methods:

- REINFORCE
- A2C
- DDPG

REINFORCE and A2C operates on discrete action states while DDPG operates on continuous action space. The results of these algorithms evaluated on the OpenAI gym LunarLander-v2[11] and LunarLanderContinuous-v2[12] and Pendulum-v0[13] environment were taken as baselines for its subsequent topology and hyperparameter optimized versions.

A. Deep Reinforcement Learning.

Deep RL is broadly classified as:

- Value or Q Learning Method
- Policy Gradient Method

In Value Learning Methods like DQN[5] based on Markov Decision Process (MDP), the agent A explores environment $s_t, s_t \in S$; steps into next state s_{t+1} by taking action a_t ; gathers scalar reward $r_t, r_t \in \mathbb{R}$ and subsequently updates the Q value by the Bellman Equation [14] given as :

$$Q(s_t, a_t) = (1-\alpha)Q(s_t, a_t) + \alpha(r(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})) \quad (1)$$

where α is the learning rate, γ is the discount factor. the policy π_t is inferred from the Q value either by ϵ greedy or some set rules.

On the other hand, a policy gradient method directly produces the stochastic policy π (probability of taking action A) given the state s_t and network parameters θ .

$$\pi(a_t|s_t, \theta) = Pr(A = a_t|S = s_t, param = \theta) \quad (2)$$

1) **REINFORCE**: REINFORCE directly predicts the policy with the update rule:

$$\Delta\theta_t = \alpha \nabla_{\theta} \log \pi(a_t|s_t) v_t \quad (3)$$

where v_t is the value function output gathered from experience. REINFORCE was one of the first policy gradient

Input: init θ

```

foreach episode ( $s_1, a_1, r_1 \dots s_{T-1}, a_{T-1}, r_{T-1}$ 
  sampled from policy  $\pi_{\theta}$ ) do
  for  $t=1$  to  $T-1$  do
     $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t|s_t) v_t$ 
  end
end

```

Algorithm 1: Pseudocode of REINFORCE.

methods which could directly predict a policy rather than infer policy from a value function approximation.

Hyperparameter:

- Learning Rate α

2) **A2C: Advantage Actor Critic method**: A2C is the synchronous version of the The Asynchronous Advantage Actor Critic method (A3C) [15]. Actor Critic methods are basically temporal difference methods with a unique memory structure to show policy irrespective of the value function. The policy network is called the actor while the value network is the critic. The critic network evaluates the effectiveness of the action taken by the actor. It is a model free on-policy learning method, where the critic evaluates a TD error given as:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (4)$$

where, V is the present value function. For a positive TD error, the taken policy is strengthened and for negative TD error, the policy prediction is weakened. This error is used to learn the weights in both actor and critic network.

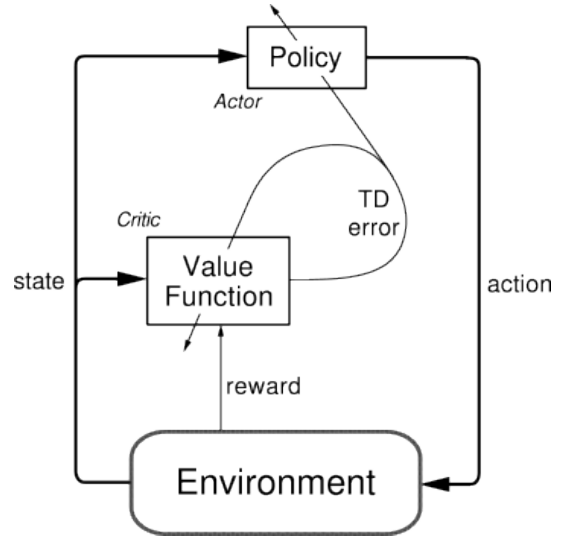


Fig. 1: Actor Critic architecture.

Hyperparameter:

- Learning Rate α for Actor and Critic
- Batch Size
- Reward Scale
- Hidden layer number for Actor and Critic
- Node length in hidden layers for Actor and Critic.

3) **DDPG: Deep Deterministic Policy Gradient**: DDPG[4] is an off-policy extension of the Actor Critic network architecture in which a stochastic policy to estimate a deterministic target policy. The actor network input is the present state value and predicts a softmax probability of the actions to be taken in continuous action space. The critic network inputs are the present state and the action value given by the actor. The critic is updated by the gradients received from the TD error. The TD target and loss functions are as follows:

$$y_t = r_t + \gamma Q^*(s_{t+1}, \mu^*(s_{t+1}|\theta^{\mu^*})|\theta^Q) \quad (5)$$

$$Loss = 1/N \sum (y_t - Q(s_t, a_t|\theta^Q))^2 \quad (6)$$

where, N is the minibatch size. DDPG has the same techniques like Experience Replay and separate target network as in DQN. Action noise may be incorporated to the action or to the topology parameter space. Most commonly used noise is the Ornstein-Uhlenbeck Noise.

Hyperparameter:

- Learning Rate α for Actor and Critic
- Batch Size
- Hidden layer number for Actor and Critic
- Node length in hidden layers for Actor and Critic.

II. EVOLUTIONARY ALGORITHMS FOR HYPERPARAMETER OPTIMIZATION: EXPERIMENT RESULTS

The Evolutionary Algorithms used in this study for hyperparameter and topology evolution are JADE[6] and Particle Swarm Optimization[9]. The baseline algorithms were run for

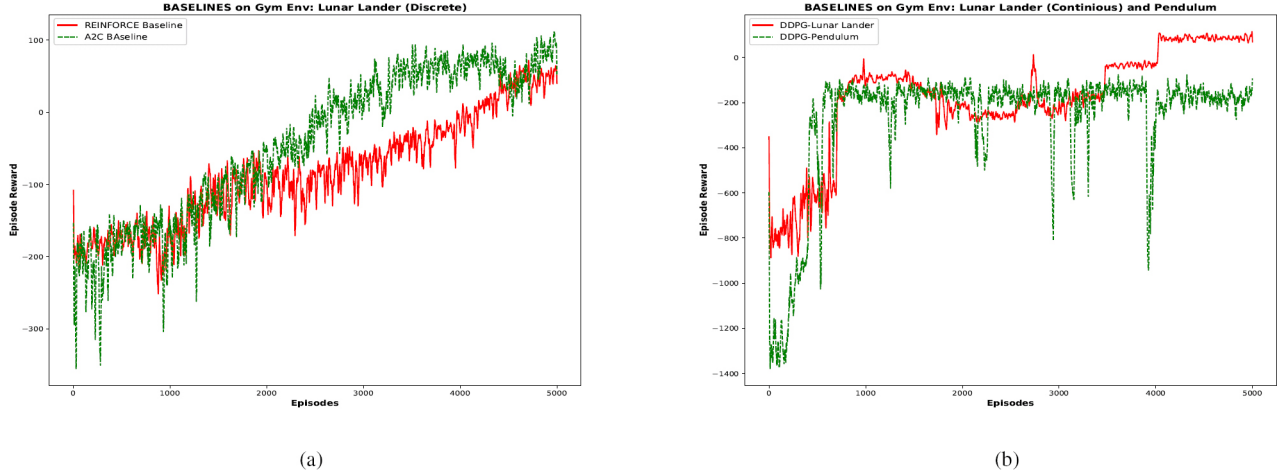


Fig. 2: Baseline for (a) REINFORCE and A2C on Gym Environment: LunarLander-v2 (Discrete). (b) DDPG on Gym Environment: LunarLanderContinuous-v2 and Pendulum-v0

Input: init Critic $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$
 init target network weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$
foreach I, M **do**
 init random Noise $\mathcal{N}$
 get init state s_t
 for $t=1, T$ **do**
 select action $a_t = \mu(s|\theta^\mu) + \mathcal{N}$
 execute action a_t , observe reward r_t and new state s_{t+1}
 store transition $[a_t, s_t, s_{t+1}, r_t]$ in buffer R
 sample a mini batch from R
 let $y_t = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1}|\theta^{\mu'}))|\theta^{Q'}$
 update critic by loss:
 $Loss = 1/N \sum (y_t - Q(s_t, a_t|\theta^Q))^2$ update
 actor by policy gradient:
 $\nabla_{\theta^\mu} J \approx (1/N) \sum \nabla_a Q(s, a|\theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_t}$
 update targets:
 $\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$
 $\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$
 end
end

Algorithm 2: Pseudocode of DDPG.

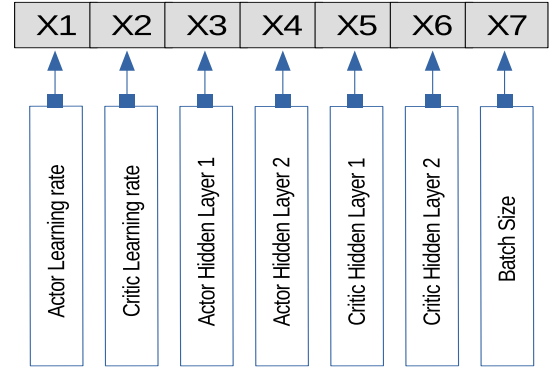


Fig. 3: The solution vector for JADE and PSO for DDPG optimization.

	REINFORCE	A2C	DDPG
Actor Topology	8,10,10,4	8,128,4	8,400,300,2
Critic Topology	-	8,128,1	10,400,300,1
Actor LR	0.0001	0.0001	0.0001
Critic LR	-	0.001	0.001
Batch Size	-	64	64

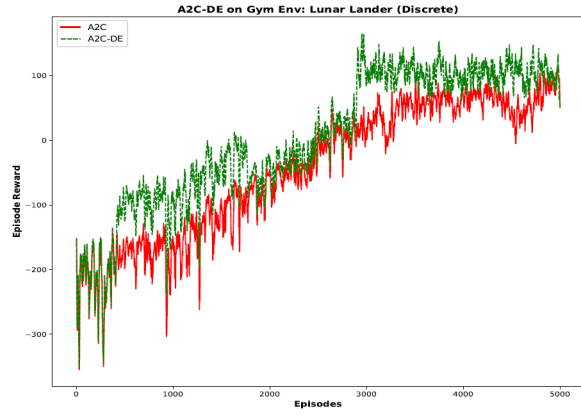
TABLE I: Initial baseline parameters

a total of 5000 episodes. The plots of the same are in Fig.2. The solution vector for JADE and PSO is shown in Fig.3. The objective/reward function for both JADE and PSO is given as :

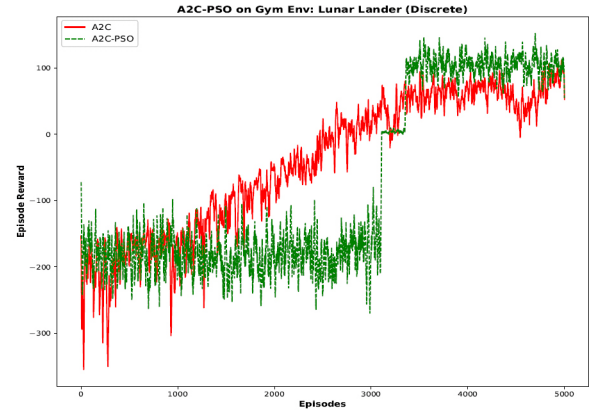
$$f(X) = \left(0.91 \left(\sum_{i=3}^6 X_i \right) + 0.1 \left(\sum_{i=1}^2 X_i \right) + X_7 \right) / R_X \quad (7)$$

where, R_X is the reward achieved for that solution vector.

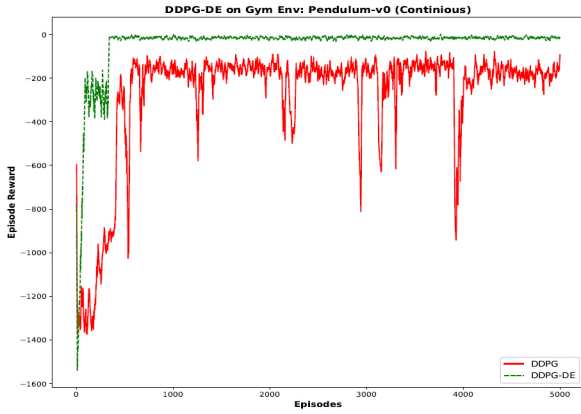
The results of topology evolution and hyperparameter optimization of A2C and DDPG by JADE and PSO are shown in Fig.4. Fig.4[a] shows the performance of JADE optimized A2C on LunarLander-v2 environment of gym which is a discrete action space problem. Fig.4[b] Fig.4[a] shows the performance of PSO optimized A2C on LunarLander-v2 environment. It is quite evident from the results that JADE optimized A2C not only shows better reward over iterations, but also has the tendency to optimize the topology of the Actor/critic networks



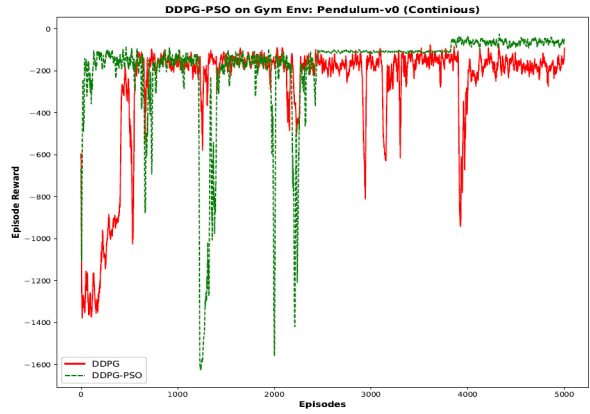
(a)



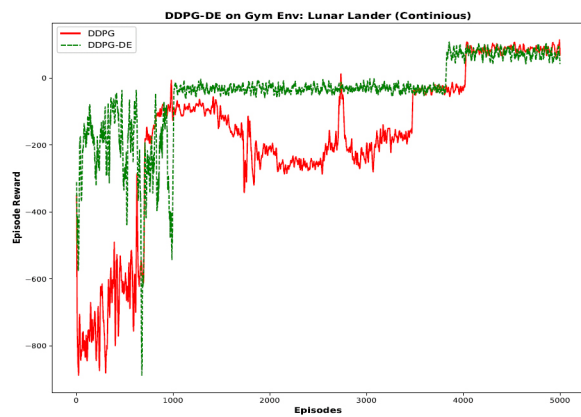
(b)



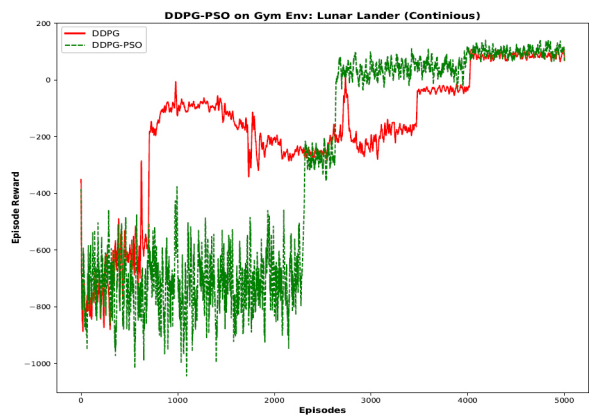
(c)



(d)



(e)



(f)

Fig. 4: (a) JADE optimized A2C on Gym Environment: LunarLander-v2 (Discrete). (b) PSO optimized A2C on Gym Environment: LunarLander-v2 (Discrete) (c) JADE optimized DDPG on Gym Environment: Pendulum-v0. (d) PSO optimized DDPG on Gym Environment: Pendulum-v0. (e) JADE optimized DDPG on Gym Environment: LunarLanderContinuous. (f) PSO optimized DDPG on Gym Environment: LunarLanderContinuous.

	DDPG-Baseline	DDPG-DE	DDPG-PSO
Actor Topology	8,128,119,2	8,128,119,2	8,238,198,2
Critic Topology	10,128,119,1	10,128,119,1	10,238,198,1
Actor LR	0.0049	0.0049	0.0016
Critic LR	0.018	0.018	0.0116
Batch Size	64	64	64

TABLE II: JADE and PSO optimised topology and hyperparameter for DDPG. Total iteration:5000.

to minimize overall trainable parameters. PSO on the other hand when applied to evolve the topology of A2C performs sub par in the initial iterations. It turns to get caught in local minimas and struggles to get out of the local minima trap. After a succinct amount of iterations, PSO manages to break the trap and starts accumulating rewards simillar to JADE. So, if iterations or its subsequent computational complexity is not an issue for a certain problem, PSO can be a viable solution; although, JADE provides better rewards in less iterations with far less computational overhead.

Fig.4[c] and [d] depicts the performance of JADE and PSO for the optimization of topology for DDPG on a simple Pendulum-v0 environment from gym.it is to be noted that the action dimension for this problem is only one.Hence though, the action space is continious in nature; it should not be a problem for DDPG to gather best rewards in less itertions by itself. Optimality of topolgy in this problem is not an issue, as any any small fully conected network for actor and critic would suffice. Nevertheless, JADE when used to optimize the topology, showed 10% increase in cumulative reward over vanilla DDPG. PSO on the other hand showed sub-optimal or simillar results in comparision to vanilla DDPG.

Fig.4[e],[f] shows the performance of JADE and PSO optimized DDPG on LunarLanderContinious environment from gym. This environment is a continious action space problem with with two element ction vector. Unlike the pendulum problem, this lunar lander problem is complicated enough with 8 state space and 2 action space. The vanilla DDPG takes umpteen number of iterations to reach a favourable reward. JADE optimized DDPG reached th same final reward of vanilla DDPG but with less iterations. PSO o the other; showed simillar performance as it showed in its LUNAR LANDER discrete optimization problem.Initially, it struggled with local minima trap but finally it reached the reward of vanilla DDPG.

JADE as a variant of Differential Evolution proves to be reaching the final termination reward of all environments both discrete and continious action space. However , in comparision PSO does not provide a computationally economical method to topology evolution and hyperparameter optimization.Table.II shows the final optimized topology and hyperparameters for DDPG optimized by JADE and PSO in continious action space.

III. CONCLUSION

It is evident from the results that hyperparameter tuning and topology evolution by evolutionary algorithms was essential

in increasing the reward over time. For discrete action space scenarios, the hyperparameter tuning alone was sufficient to increase the reward. JADE was a clear choice for hyperparameter and topology optimization in discrete action space problems. JADE was again used to optimize hyperparameters of the continious domain DDPG network with good results. However on using JADE to optimize the topology of DDPG, the reward rarely increased. PSO optimized DDPG hardly showed any improvement. We can safely conclude that hyperparameter and topology optimization by JADE or any other variants of Differential Evolution on discrete action space policy gradient reinforcement learning methods bear good results; thus eliminating the need of hand tuning parameters.

REFERENCES

- [1] W. Wang, J. Yang, J. Xiao, S. Li, and D. Zhou, "Face recognition based on deep learning," in *Human Centered Computing*, Q. Zu, B. Hu, N. Gu, and S. Seng, Eds. Cham: Springer International Publishing, 2015, pp. 812–820.
- [2] X. Shi, S. Tian, L. Yu, L. Li, and S. Gao, "Prediction of soil adsorption coefficient based on deep recursive neural network," *Automatic Control and Computer Sciences*, vol. 51, no. 5, pp. 321–330, Sep 2017. [Online]. Available: <https://doi.org/10.3103/S0146411617050066>
- [3] Y. Yuan, L. Mou, and X. Lu, "Scene recognition by manifold regularized deep learning architecture," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 26, no. 10, pp. 2222–2233, Oct 2015.
- [4] S. Gu, T. P. Lillicrap, I. Sutskever, and S. Levine, "Continuous deep q-learning with model-based acceleration," in *ICML*, 2016.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *CoRR*, vol. abs/1312.5602, 2013. [Online]. Available: <http://arxiv.org/abs/1312.5602>
- [6] J. Zhang and A. C. Sanderson, "Jade: Adaptive differential evolution with optional external archive," *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 945–958, Oct 2009.
- [7] S. K. Anne Auger and N. Hansen, "A restart cma evolution strategy with increasing population size," in *2005 IEEE Congress on Evolutionary Computation*, vol. 2, Sept 2005, pp. 1769–1776 Vol. 2.
- [8] X. S. Yang and S. Deb, "Cuckoo search via levy flights," in *2009 World Congress on Nature Biologically Inspired Computing (NaBIC)*, Dec 2009, pp. 210–214.
- [9] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Neural Networks, 1995. Proceedings., IEEE International Conference on*, vol. 4, Nov 1995, pp. 1942–1948 vol.4.
- [10] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ser. ICML'10. USA: Omnipress, 2010, pp. 807–814. [Online]. Available: <http://dl.acm.org/citation.cfm?id=3104322.3104425>
- [11] "LunarLander-v2," <https://gym.openai.com/envs/LunarLander-v2/>.
- [12] "LunarLanderContinious-v2," <https://gym.openai.com/envs/LunarLanderContinious-v2/>.
- [13] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016, cite arxiv:1606.01540. [Online]. Available: <http://arxiv.org/abs/1606.01540>
- [14] R. Bellman, *Dynamic Programming*. Dover Publications, 1957.
- [15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous Methods for Deep Reinforcement Learning," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 2016, pp. 1928–1937. [Online]. Available: <http://proceedings.mlr.press/v48/mniha16.html>